END-TERM PROJECT-01 REPORT

ON

Car Dodging Game

B.Tech. [Computer Science and Engineering]

Semester: 6th

Section: 23412K1

Course Title & Code: Game Programming with C++ (CSE 3545)

Submitted By:

Student Name: Amit Kumar

Registration No: 2341011129

Guided By: Ishan Ayus

Academic Year: 2025 - 26

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING AND TECHNOLOGY, INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY

BHUBANESWAR, ODISHA, INDIA

ABSTRACT

This project focuses on the design and development of a 2D car dodging game implemented using C++ and the SFML (Simple and Fast Multimedia Library). The main objective of the game is to control a player car and avoid collisions with continuously spawning enemy vehicles on a moving road.

The project demonstrates important Object-Oriented Programming concepts such as inheritance, abstraction, and polymorphism through the use of multiple classes including Car, PlayerCar, and EnemyCar. The game includes real-time movement, collision detection, scoring mechanisms, increasing difficulty levels, and sound integration.

The game was successfully implemented and tested, providing smooth gameplay with responsive controls and dynamic enemy generation.

Submitted By

Amit Kumar

Guided By

Ishan Ayus

TABLE OF CONTENTS

LIST OF FIGURES

01. Introduction

C++ is one of the most powerful and widely used programming languages in computer science, especially in the field of system programming and game development. It provides strong support for object-oriented programming concepts such as classes, inheritance, polymorphism, and encapsulation.

This practical project focuses on the development of a Car Dodging Game using C++ along with the SFML (Simple and Fast Multimedia Library). The game is designed to provide an interactive graphical environment where the player controls a car and avoids incoming enemy vehicles.

The project helps in improving logical thinking, problem-solving ability, and understanding of real-time programming through hands-on implementation. It also introduces important game development concepts such as rendering, event handling, collision detection, and game state management.

The main aim of this project is to apply theoretical knowledge of C++ programming into a working real-time application and gain practical experience in building interactive games.

02. Problem Statement

The problem addressed in this project is to design and implement a real-time car dodging game using C++ that satisfies the following requirements:

- Smooth player movement using keyboard controls
- Random enemy car spawning
- Collision detection between player and enemy vehicles
- Score tracking system
- Increasing difficulty during gameplay
- Use of Object-Oriented Programming concepts
- Efficient rendering using SFML graphics library

The game should provide responsive gameplay and maintain smooth performance during execution.

03. Objectives of the Project

The objectives of this project are defined to provide a clear direction for the design and development of the 2D Car Dodging Game. The objectives of this practical project are:

- To develop a functional 2D car dodging game using C++
- To implement Object-Oriented Programming concepts
- To use inheritance and polymorphism through class hierarchy
- To implement collision detection mechanisms
- To create a real-time scoring system
- To improve programming and debugging skills
- To understand game loops and frame-based updates
- To gain practical experience using the SFML multimedia library

To gain practical exposure to real-time programming applications.

04. Tools and Technologies Used

The following tools and technologies were used in this project:

Programming Language: C++, SFML

Compiler: G++ / Turbo C / Dev-C++

Development Environment: Text Editor / VS Code

Operating System: Windows / Linux

These tools provide a suitable environment for writing, compiling, and executing C programs.

05. Methodology

The development of the 2D Car Dodging Game followed a structured software engineering and game design lifecycle to ensure clean code and smooth gameplay. The steps included:

- **Requirement Analysis & Conceptualization:** Defining the core gameplay mechanics, such as the player's movement constraints, the randomized spawning of enemy vehicles, and the continuous scoring system.

- **Architectural Design:** Planning the Object-Oriented structure. This involved designing modular C++ classes for the player's car and the enemy vehicles to keep the code organized and reusable.

- **Implementation & Graphics Integration:** Writing the core logic in C++ and utilizing the SFML library to handle rendering 2D sprites, drawing the game window, and capturing real-time keyboard inputs.

- **Physics & Collision Logic:** Implementing frame-independent movement (using SFML's time delta) so the cars move smoothly, and creating bounding-box algorithms to instantly detect when the player crashes into oncoming traffic.

- **Testing and Debugging:** Running the game through various scenarios to tune the hitboxes for fair gameplay, ensuring off-screen cars are properly recycled to prevent memory leaks, and resolving linker errors.

- **Final Playtesting:** Verifying that the game's difficulty dynamically scales over time and ensuring a stable, lag-free experience for the end user.

06. Results Analysis and Screenshots

The 2D Car Dodging Game was successfully compiled and executed, achieving the intended gameplay mechanics without runtime errors or memory leaks. The integration of C++ with the SFML library resulted in a stable, responsive application that accurately processes real-time user inputs.

Key observations from the final execution include:

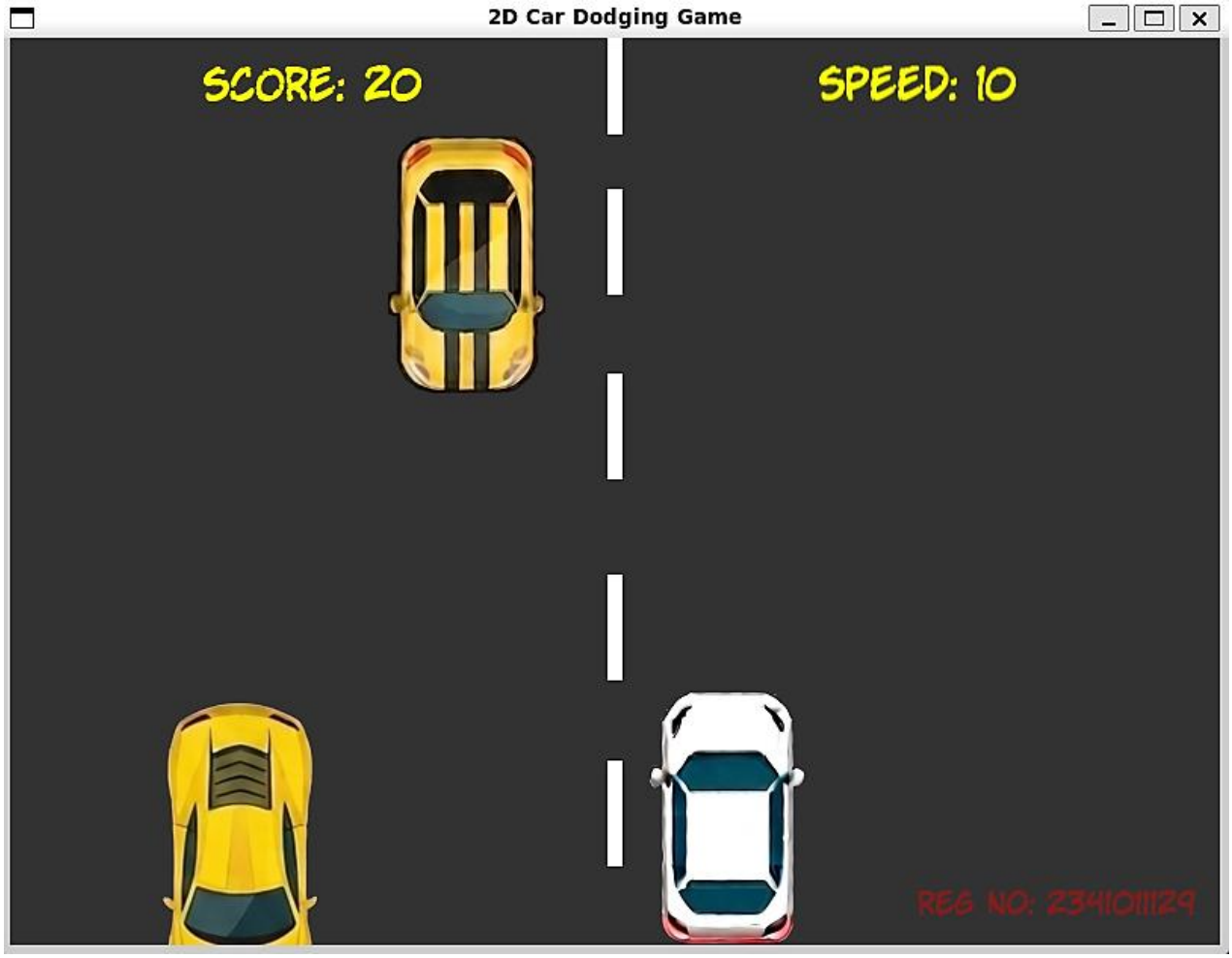
- **Visual Rendering:** The game window initializes correctly, drawing the road background, the player's car, and the enemy sprites smoothly at a consistent frame rate (60 FPS).
- **Input Responsiveness:** The player's vehicle responds instantaneously to left and right keyboard commands, strictly adhering to the defined road boundaries without moving off-screen.
- **Dynamic Generation:** Enemy vehicles spawn continuously at randomized horizontal positions, ensuring no two playthroughs are identical.
- **Collision Detection & State Management:** The bounding-box intersection logic works flawlessly. When the player's car overlaps with an enemy vehicle, the game loop instantly transitions to the "Game Over" state, halting movement and displaying the final score.
- **Scoring System:** The real-time score tracking updates accurately based on the player's survival time and successfully dodged traffic.

Output Screenshots:

Figure 1 (Home Screen / Start State):

The image shows the start screen of the 2D Car Dodging Game. The player's car is placed at the bottom of the road, and the message "PRESS ENTER TO START" is displayed to begin the game.

Figure 2 (Playing State):



The image shows the gameplay screen where the player controls the white car and avoids other moving cars. The score and speed are displayed at the top.

Figure 3 (Paused State):

The image shows the game in paused mode. The message “GAME IS PAUSED” appears at the center while all cars stop moving temporarily.

Figure 4 (Game Over State):

The image shows the game over screen after the player's car collides with another car. The player can press Enter to restart the game.

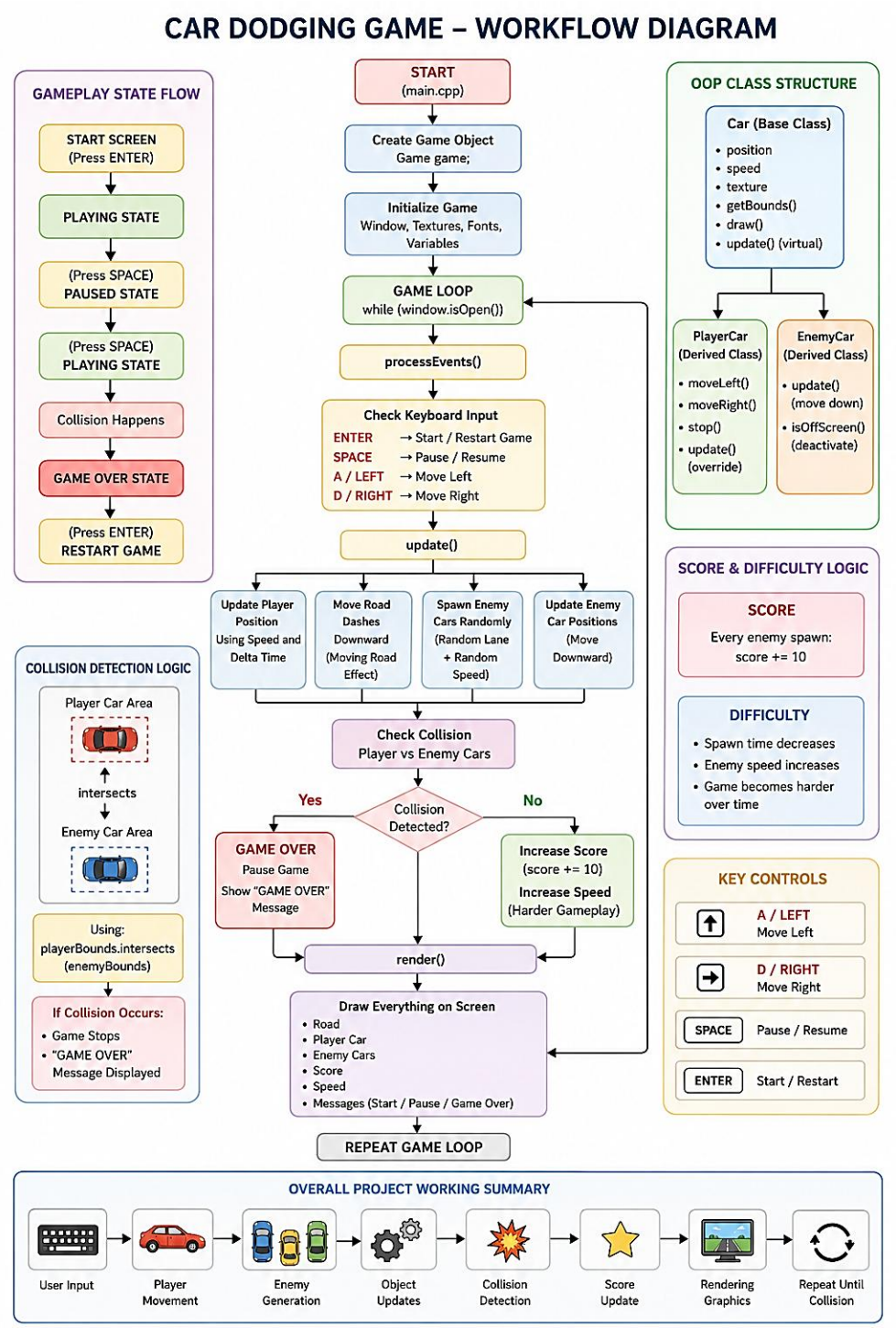
07. Logical Understanding

The game logic is based on the following concepts:

- The player car moves horizontally based on keyboard input.
- Enemy cars continuously move downward.
- Enemy cars are generated randomly in lanes.
- Collision detection checks intersection between player and enemy cars.
- The score increases when the player successfully avoids enemies.
- Game speed increases gradually over time.

This project helps in understanding the flow of control and logical execution in C programming.

Figure 5 (Workflow Diagram):



The flow diagram explains the complete working of the 2D Car Dodging Game developed using C++ and SFML. The game starts by initializing the game window, textures, fonts, and variables. The player controls the car using keyboard inputs while enemy cars spawn randomly and move downward continuously. The game checks for collisions between the player and enemy cars. If a collision occurs, the game enters the Game Over state. During gameplay, the score and speed increase gradually to make the game more challenging. Finally, the render function displays all game objects such as the road, cars, score, and messages on the screen.

08. Conclusion

In conclusion, The Car Dodger Game was successfully designed and implemented using C++ and SFML. The project helped in understanding important concepts of Object-Oriented Programming, game loops, collision detection, and real-time graphics rendering.

The game provides smooth gameplay with proper object management and efficient collision handling. The project also improved practical coding, debugging, and problem-solving abilities.

Overall, the project successfully achieved all intended objectives.

09. Future Scopes and Application

Future improvements that can be added to this project include:

- Multiple levels and advanced difficulty modes
- Power-ups and bonus items
- Leaderboard system using file handling
- Multiplayer gameplay support
- Improved graphics and animations
- Mobile platform support
- AI-controlled enemy vehicles

This project can be further extended into a complete arcade racing game.

References

(1) Practical C Programming by B.M. Harwani (Packt)

Annexure

The following section includes the code of the game including the explanation for different portions.

Game.cpp

```
#include "Game.h"
#include <sstream>
#include <cstdlib>

Car::Car(float startX, float startY, float speed) : m_Speed(speed){
   m_Position.x = startX;
   m_Position.y = startY;
}

void Car::draw(RenderWindow &window) {
   window.draw(m_Shape);
}
FloatRect Car::getBounds() const {
   return m_Shape.getGlobalBounds();
}
Vector2f Car::getPosition() const {
   return m_Position;
}

PlayerCar::PlayerCar(float startX, float startY) : Car(startX, startY, 400.0f){
   m_Shape.setPosition(m_Position);
}

void PlayerCar::setTexture(const Texture &texture) {
   m_Shape.setTexture(texture);
}
```

```
void PlayerCar::moveLeft() {
    m_MovingLeft = true;
}
void PlayerCar::moveRight() {
    m_MovingRight = true;
}
void PlayerCar::stopLeft() {
    m_MovingLeft = false;
}
void PlayerCar::stopRight() {
    m_MovingRight = false;
}
void PlayerCar::update(Time dt){
    if (m_MovingLeft)
        m_Position.x -= m_Speed * dt.asSeconds();
    if (m_MovingRight)
        m_Position.x += m_Speed * dt.asSeconds();
    float carWidth = m_Shape.getGlobalBounds().width;

    if (m_Position.x < 0)
        m_Position.x = 0;
    if (m_Position.x > 800.f - carWidth)
        m_Position.x = 800.f - carWidth;

    m_Shape.setPosition(m_Position);
}

EnemyCar::EnemyCar() : Car(0.f, 0.f, 0.f), isActive(false) {}

void EnemyCar::spawn(float startX, float startY, float speed, const Texture &texture){
    m_Shape.setTexture(texture, true);
```

```
    float carWidth = m_Shape.getGlobalBounds().width;

    if (startX > 800.f - carWidth){
        startX = 800.f - carWidth;
    }

    m_Position.x = startX;
    m_Position.y = startY;
    m_Speed = speed;
    isActive = true;
    m_Shape.setPosition(m_Position);
}

void EnemyCar::update(Time dt){
    if (!isActive)
        return;
    m_Position.y += m_Speed * dt.asSeconds();
    m_Shape.setPosition(m_Position);
}

void Game::centerText(Text &text, float x, float y){
    FloatRect details = text.getLocalBounds();
    text.setOrigin(details.left + details.width / 2.0f, details.top + details.height / 2.0f);
    text.setPosition(x, y);
}

Game::Game() : window(VideoMode(800, 600), "2D Car Dodging Game"),
               player(375.f, 430.f), score(0),
               currentSpeedMultiplier(1.0f), spawnTimer(0.f),
               spawnThreshold(1.5f), isGameOver(false),
               gamestart(false), pause(true), acceptInput(true)
{
```

```
    window.setFramerateLimit(60);
    for (int i = 0; i < NUM_DASHES; i++){
        roadDashes[i].setSize(Vector2f(10.f, 70.f));
        roadDashes[i].setFillColor(Color::White);
        roadDashes[i].setPosition(395.f, i * 130.f);
    }

    font.loadFromFile("/mnt/g/wsl/GPWC/CarDodgingGame/Font/KOMIKAP_.ttf");

    scoreText.setFont(font);
    scoreText.setCharacterSize(28);
    scoreText.setFillColor(Color::Yellow);

    speedText.setFont(font);
    speedText.setCharacterSize(28);
    speedText.setFillColor(Color::Yellow);

    messageText.setFont(font);
    messageText.setCharacterSize(40);
    messageText.setFillColor(Color::White);
    messageText.setString("Press Enter to start !!!!");
    centerText(messageText, 400.f, 300.f);

    watermarkText.setFont(font);
    watermarkText.setString("Reg No: 2341011129");
    watermarkText.setCharacterSize(20);
    watermarkText.setFillColor(Color(255, 0, 0, 100));
    watermarkText.setPosition(600.f, 560.f);


playerTexture.loadFromFile("/mnt/g/wsl/GPWC/CarDodgingGame/Graphics/WhiteCar.png"
);
```

```
    player.setTexture(playerTexture);


enemyTextures[0].loadFromFile("/mnt/g/wsl/GPWC/CarDodgingGame/Graphics/RedCar1.p
ng");

enemyTextures[1].loadFromFile("/mnt/g/wsl/GPWC/CarDodgingGame/Graphics/RedCar2.p
ng");

enemyTextures[2].loadFromFile("/mnt/g/wsl/GPWC/CarDodgingGame/Graphics/YellowCar
1.png");

enemyTextures[3].loadFromFile("/mnt/g/wsl/GPWC/CarDodgingGame/Graphics/YellowCar
2.png");

enemyTextures[4].loadFromFile("/mnt/g/wsl/GPWC/CarDodgingGame/Graphics/YellowCar
3.png");
}


void Game::run(){
    Clock clock;
    while (window.isOpen()){
        processEvents();
        Time dt = clock.restart();
        update(dt);
        render();
    }
}


void Game::processEvents(){
    Event event;
    while (window.pollEvent(event)){
        if (event.type == Event::Closed)
            window.close();
        if (event.type == Event::KeyReleased && !acceptInput){
```

```
        acceptInput = true;
    }
}

if (acceptInput){
    if (Keyboard::isKeyPressed(Keyboard::Enter)){
        if (isGameOver){
            resetGame();
        }
        else{
            gamestart = true;
            pause = false;
            messageText.setString("");
        }
        acceptInput = false;
    }

    if (Keyboard::isKeyPressed(Keyboard::Space) && gamestart && !isGameOver){
        pause = !pause;
        messageText.setString(pause ? "Game is Paused !!!" : "");
        centerText(messageText, 400.f, 300.f);
        acceptInput = false;
    }
}

if (!pause && gamestart && !isGameOver){
    if (Keyboard::isKeyPressed(Keyboard::Left) || Keyboard::isKeyPressed(Keyboard::A))
        player.moveLeft();
    else
        player.stopLeft();

    if (Keyboard::isKeyPressed(Keyboard::Right) || Keyboard::isKeyPressed(Keyboard::D))
```

```
            player.moveRight();
        else
            player.stopRight();
    }
    else{
        player.stopLeft();
        player.stopRight();
    }
}

void Game::update(Time dt){
    if (pause || !gamestart || isGameOver)
        return;
    player.update(dt);
    float roadSpeed = 300.f * currentSpeedMultiplier;
    for (int i = 0; i < NUM_DASHES; i++){
        roadDashes[i].move(0.f, roadSpeed * dt.asSeconds());
        if (roadDashes[i].getPosition().y > 600.f)
            roadDashes[i].setPosition(395.f, -100.f);
    }

    spawnTimer += dt.asSeconds();
    if (spawnTimer >= spawnThreshold){
        spawnEnemy();
        spawnTimer = 0.f;
        score += 10;

        if (spawnThreshold > 0.4f)
            spawnThreshold -= 0.05f;
        currentSpeedMultiplier += 0.05f;
    }
```

```
    for (int i = 0; i < MAX_ENEMIES; i++){
        if (enemies[i].isActive){
            enemies[i].update(dt);

            if (enemies[i].getPosition().y > 600.f){
                enemies[i].isActive = false;
            }
        }
    }
    checkCollisions();

    scoreText.setString("SCORE: " + std::to_string(score));
    centerText(scoreText, 200.f, 30.f);

    speedText.setString("SPEED: " + std::to_string((int)(currentSpeedMultiplier * 10)));
    centerText(speedText, 600.f, 30.f);
}

void Game::spawnEnemy(){
    int lane = rand() % 4;
    float startX = 100.f + (lane * 150.f);
    float randomSpeed = (200.f + (rand() % 100)) * currentSpeedMultiplier;
    int randomCarImage = rand() % 5;

    for (int i = 0; i < MAX_ENEMIES; i++){
        if (!enemies[i].isActive){
            enemies[i].spawn(startX, -100.f, randomSpeed, enemyTextures[randomCarImage]);
            break;
        }
    }
}
```

```
void Game::checkCollisions(){
    FloatRect playerBounds = player.getBounds();
    for (int i = 0; i < MAX_ENEMIES; i++){
        if (enemies[i].isActive && playerBounds.intersects(enemies[i].getBounds())){
            isGameOver = true;
            pause = true;
            gamestart = false;

            messageText.setString("GAME OVER !!!!!\nPress Enter to restart");
            centerText(messageText, 400.f, 300.f);
        }
    }
}

void Game::render(){
    window.clear(Color(50, 50, 50));
    for (int i = 0; i < NUM_DASHES; i++)
        window.draw(roadDashes[i]);

    player.draw(window);
    for (int i = 0; i < MAX_ENEMIES; i++){
        if (enemies[i].isActive)
            enemies[i].draw(window);
    }

    window.draw(scoreText);
    window.draw(speedText);

    if (!gamestart || pause || isGameOver)
        window.draw(messageText);

    window.draw(watermarkText);
```

```
    window.display();
}

void Game::resetGame(){
    score = 0;
    currentSpeedMultiplier = 1.0f;
    spawnThreshold = 1.5f;

    for (int i = 0; i < MAX_ENEMIES; i++)
        enemies[i].isActive = false;

    isGameOver = false;
    gamestart = true;
    pause = false;
    messageText.setString("");

    player = PlayerCar(375.f, 430.f);
    player.setTexture(playerTexture);
}
```

Game.h

```
#pragma once
#include <SFML/Graphics.hpp>
using namespace sf;

class Car{
protected:
    Sprite m_Shape;
    Vector2f m_Position;
    float m_Speed;
```

```
public:
    Car(float startX, float startY, float speed);
    virtual ~Car() = default;

    virtual void update(Time dt) = 0;

    void draw(RenderWindow &window);
    FloatRect getBounds() const;
    Vector2f getPosition() const;
};

class PlayerCar : public Car{
private:
    bool m_MovingLeft = false;
    bool m_MovingRight = false;

public:
    PlayerCar(float startX, float startY);

    void setTexture(const Texture &texture);
    void moveLeft();
    void moveRight();
    void stopLeft();
    void stopRight();

    void update(Time dt) override;
};

class EnemyCar : public Car{
public:
    bool isActive;
```

```
    EnemyCar();

    void spawn(float startX, float startY, float speed, const Texture &texture);
    void update(Time dt) override;
};

class Game{
private:
    RenderWindow window;

    Texture playerTexture;
    Texture enemyTextures[5];

    PlayerCar player;

    static const int MAX_ENEMIES = 10;
    EnemyCar enemies[MAX_ENEMIES];

    Font font;
    Text scoreText;
    Text speedText;
    Text messageText;
    Text watermarkText;

    static const int NUM_DASHES = 6;
    RectangleShape roadDashes[NUM_DASHES];

    int score;
    float currentSpeedMultiplier;
    float spawnTimer;
    float spawnThreshold;
```

```
    bool gamestart;
    bool pause;
    bool isGameOver;
    bool acceptInput;

    void processEvents();
    void update(Time dt);
    void render();
    void spawnEnemy();
    void checkCollisions();
    void resetGame();
    void centerText(Text &text, float x, float y);

public:
    Game();
    void run();
};
```

Main.cpp

```
#include "Game.h"
#include <ctime>
#include <cstdlib>

int main(){
    srand(static_cast<unsigned>(time(nullptr)));
    Game game;
    game.run();
    return 0;
}
```